Machine learning course report

מגיש: גיא סהר 206779480

שאלה 1:

בחרתי ללכת על הדרך שהוצעה בשאלה.
חילקתי את ציר הזמן לחלונות של שניה ולכל דטא בתוך חלון זמן זה עבר תהליך של רגרסייה לינארית למציאת וקטור המשקלים.
כמו כן המיקום של הדובר משודר בצורת האות הבא אם נוריד את הניחות והרעש:

$$S_k(t) = cos(2\pi f_0 t - 2\pi f_0 \tau_k)$$

משמע המיקום מפורש בעזרת הפאזה של האות.
הכנתי פוקנציה המחשבת את הפאזה של האות המחושב עבור נקודה בציר ו סנסור מסוים:

```
def compute_phase (x_candidate, sensor_x):



    distance = np.sqrt((x_candidate - sensor_x)**2 + sensor_y**2)

    tau = distance / c

    phi = 2*np.pi*f0*tau

    return phi
```

הפונקציה מממשת את החישוב האנליטי של האות עבור כל נקודה בצירה x

הפונקציה הבאה מייצרת את מטריצת וקטור המאפיינים עבור כל החיישנים:

```
def build_feature_matrix(t_window, x_candidate):

    feature_list = []

    for k in range(3):

        phi_k = compute_phase(x_candidate, sensor_x[k])
```

```
        feature_k = np.cos(2*np.pi*f0*t_window - phi_k)
        feature_list.append(feature_k)
    N = len(t_window)
    X = np.zeros((3*N,3))

    X[:N,0] = feature_list[0]
    X[N:2*N,1] = feature_list[1]
    X[2*N:3*N,2] = feature_list[2]
    return X
```

היא משתמש בפונקציה לחישוב הפאזה ושמה את התוצאות ברשימה עבור כל חיישן ושמה את הרשימה במטריצה 3x3 כמו שרשום בהוראות עבור כל מיקום בציר.

עבור קביעת חלון הזמן כדי שיהיה נוח לייצר כמה חלונות שונים עבור ניסויים הגדרתי פונקציית עזר:

```
def set_time_windows(t, T):
  windows =[]
  t_start = t[0]
  while t_start < t[-1]:
      idx = np.where((t >= t_start) & (t < t_start + T))[0]
      if len(idx) > 0:
          windows.append(idx)
      t_start += T
  print(f"number of windows: {len(windows)}")
  return windows
```

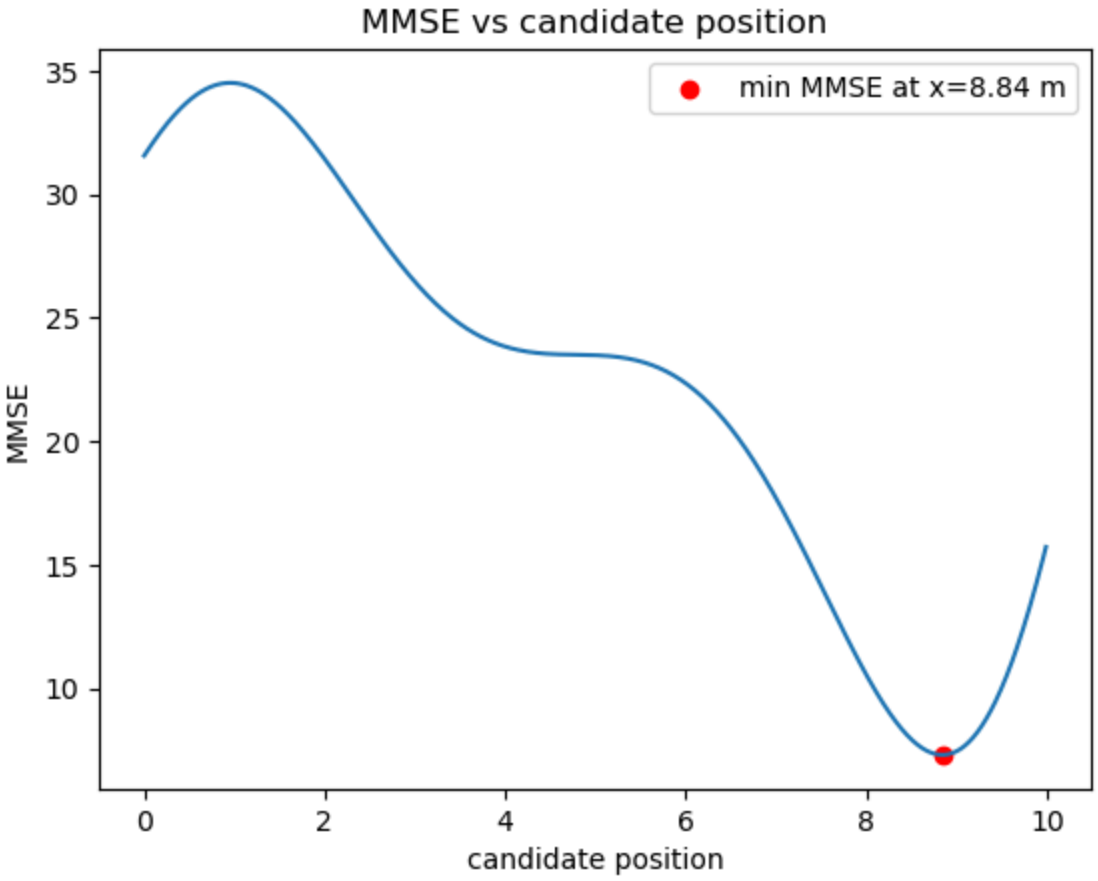
את האלגוריתם המרכזי שמתי בפונקציה שנקראת 'main' שמחשבת את וקטור המשקלים עבור כל מיקום אפשרי על הציר ובודקת מי מהמשקלים מייצר את השגיאה הקטנה ביותר ובכך קובעת איזה אות מחושב הכי מתאים

לאות הקלט בכל נקודת זמן בחלון ולבסוף מכניסה מיקום סופי שהוא מכל הערכים מי בעל השגיאה הריבועית הקטנה ביותר.

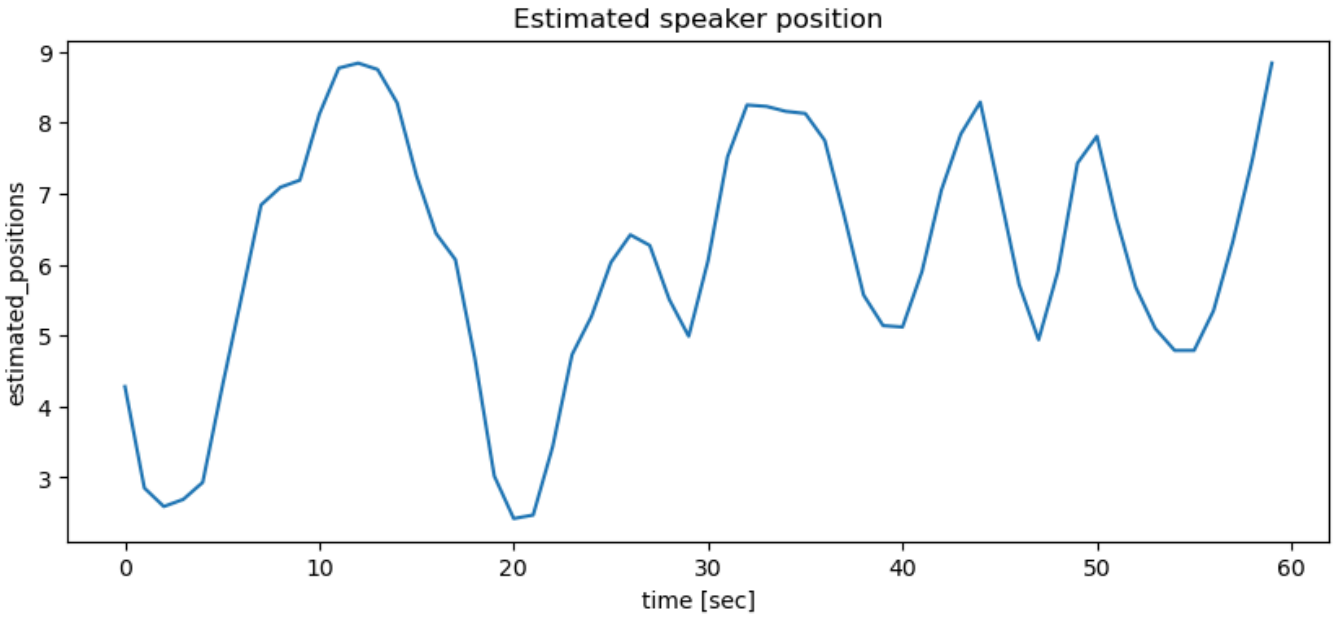
```
def main(t,s1,s2,s3,windows):

    estimated_positions = []



    for idx in windows:

        t_window = t[idx]

        s_window = np.concatenate([s1[idx], s2[idx], s3[idx]])

        mmse_values = []

        for x_c in x_candidates:

            X = build_feature_matrix(t_window, x_c)

            w = np.linalg.pinv(X) @ s_window

            error = np.linalg.norm(X @ w - s_window)**2

            mmse_values.append(error)

        best_x = x_candidates[np.argmin(mmse_values)]

        estimated_positions.append(best_x)

    min_idx = np.argmin(mmse_values)

    min_x = x_candidates[min_idx]

    min_mmse = mmse_values[min_idx]

    plt.plot(x_candidates, mmse_values)

    plt.scatter(min_x, min_mmse, color='red', label=f"min MMSE at x=
{min_x:.2f} m")

    plt.legend()
```

```
    plt.xlabel("candidate position")
    plt.ylabel("MMSE")
    plt.title("MMSE vs candidate position")
    plt.show()
    time_windows = np.arange(len(estimated_positions))* T


    plt.figure(figsize=(10,4))
    plt.plot(time_windows, estimated_positions, label="estimated position")
    plt.xlabel("time [sec]")
    plt.ylabel("estimated_positions")
    plt.title("Estimated speaker position")
    plt.show()
    return estimated_positions
```

גרף השגיאה :

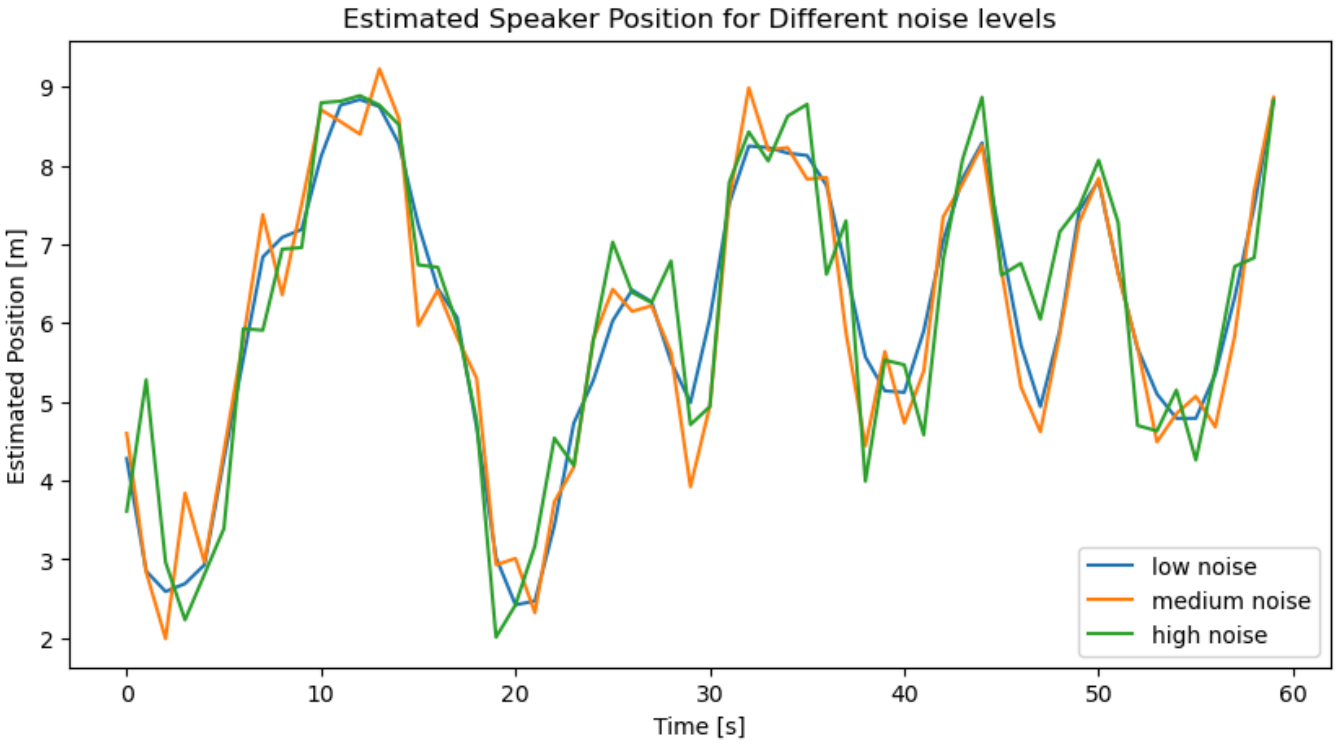


גרף מיקום הדובר בזמן:



שאלה 2,3:

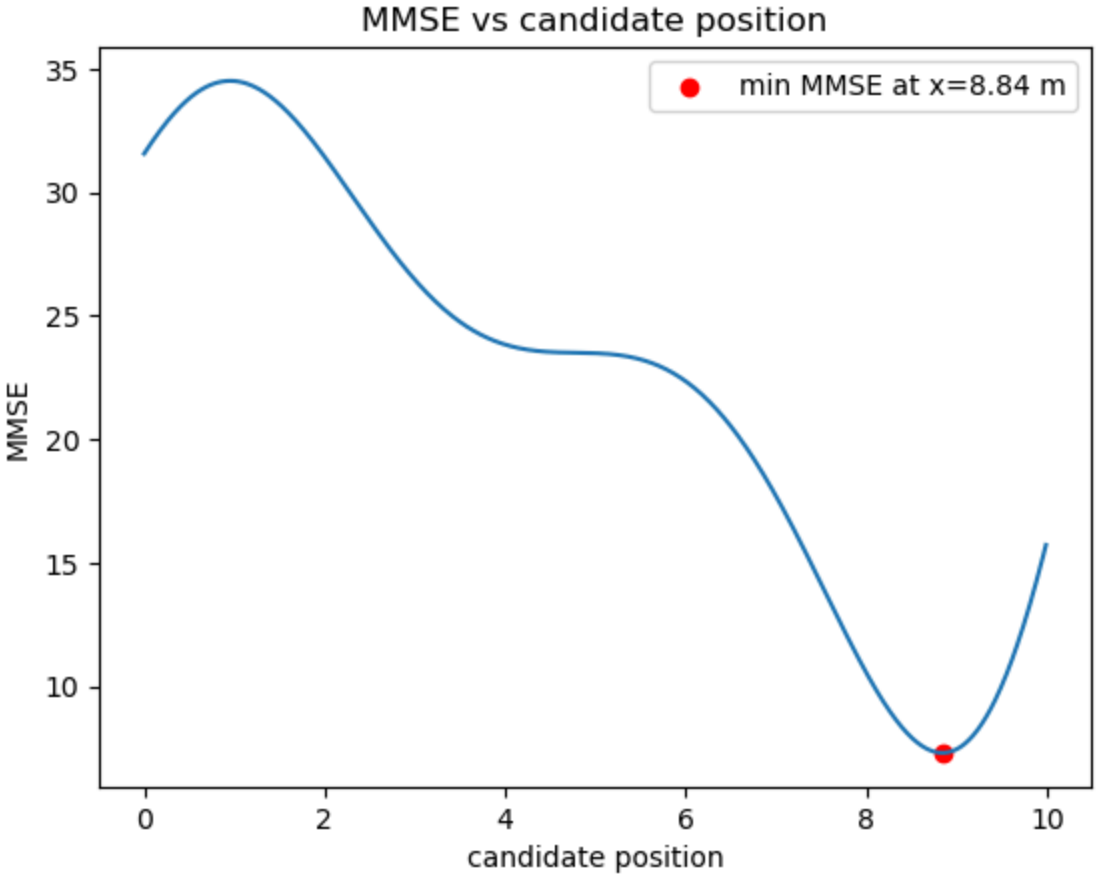
שמתי את שלושת הגרפים ביחד על ציר אחד:



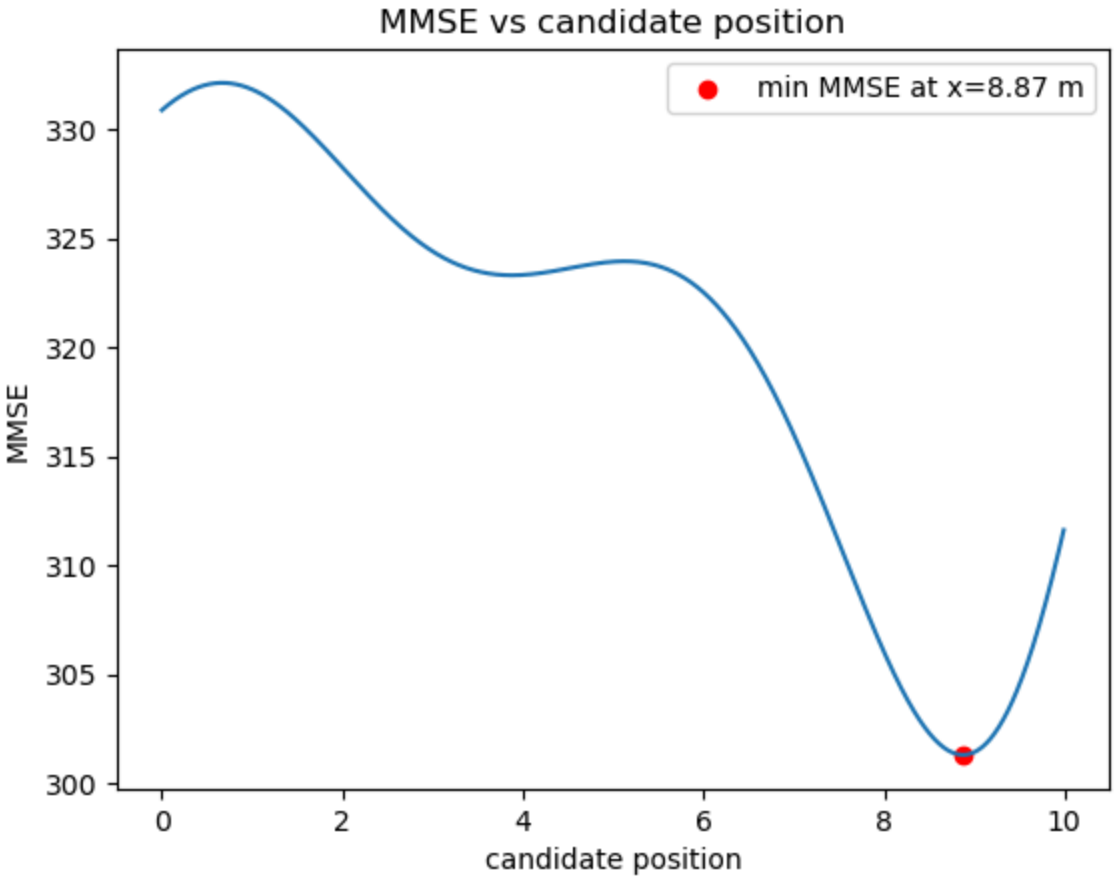
ניתן לראות בברור שהם די קרובים אבל ניתן לראות בבירור שהרעש משפיע.
הגרף של הרעש החלק חלק יותר בעוד והגרף עם הרעש הבינוני והחזק עוקבים אחריו אבל ניתן לראות קפיצות וריצוד ברור סביבו

בנוסף כאשר מסתכלים על הגרפי השגיאה בשלושת המקרים ניתן לראות כי לא הגיעו לתוצאות רחוקות מאוד במיקום אבל סדר גדול השגיאה משתנה משמעותית

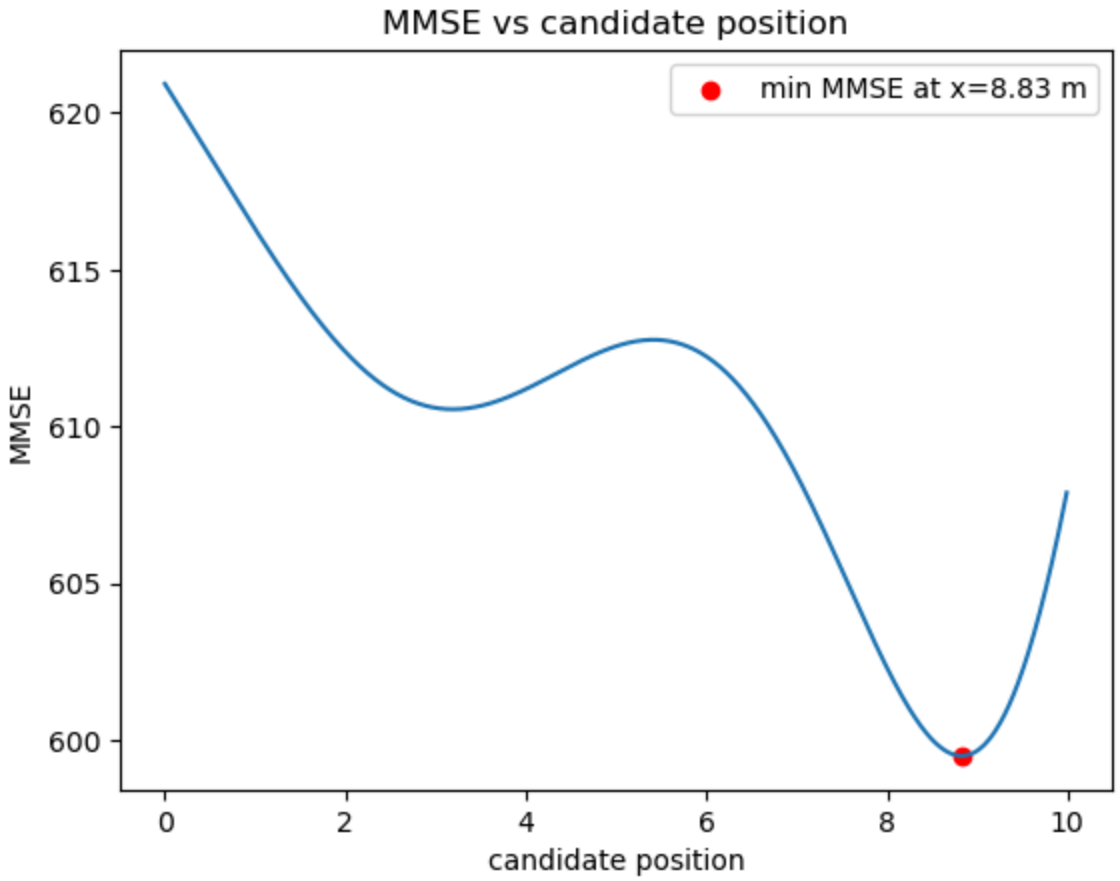
עם רעש חלש השגיאה הכי נמוכה קטנה מ 10 והגבוה באזור 35:



עם רעש בינוני השגיאה הכי נמוכה נמצאת בין 300 ל 305 והגבוה באזור 335:

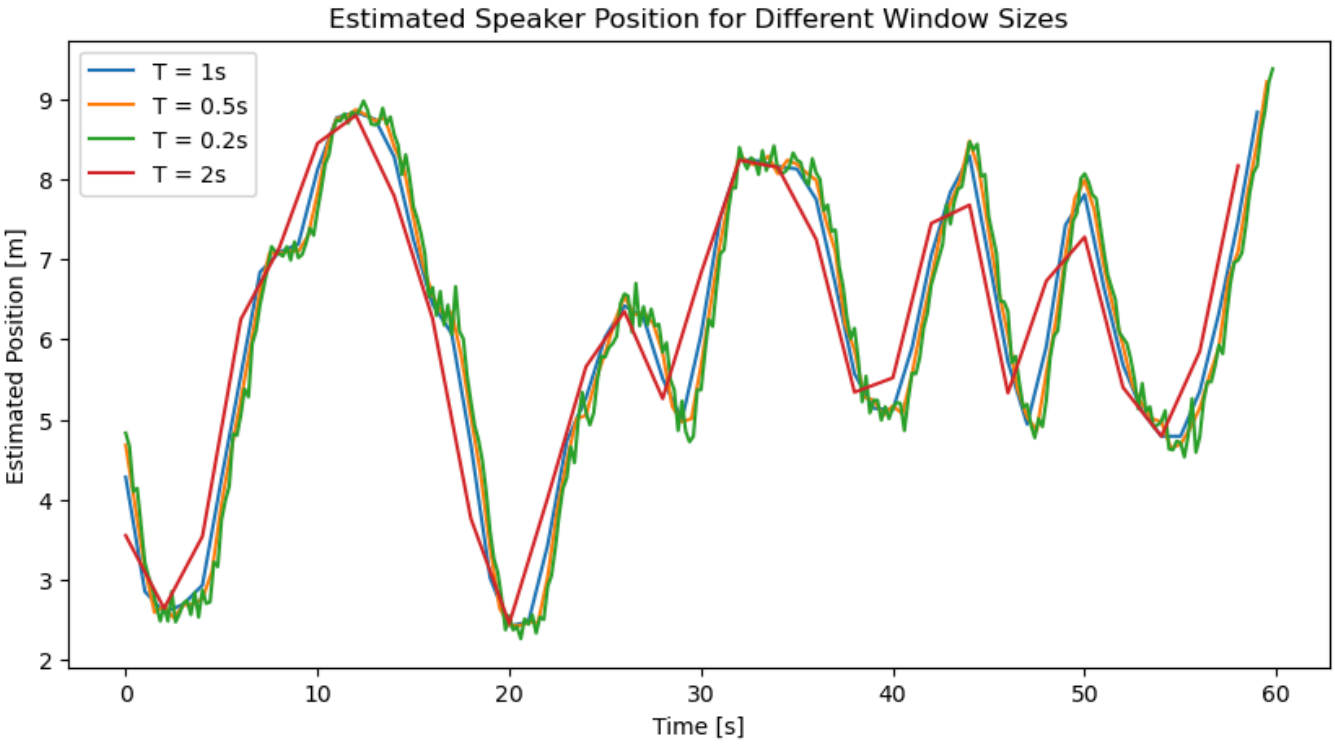


עם רעש חזק השגיאה הכי נמוכה באזור 600 והגבוה באזור 625:



שאלה 4:

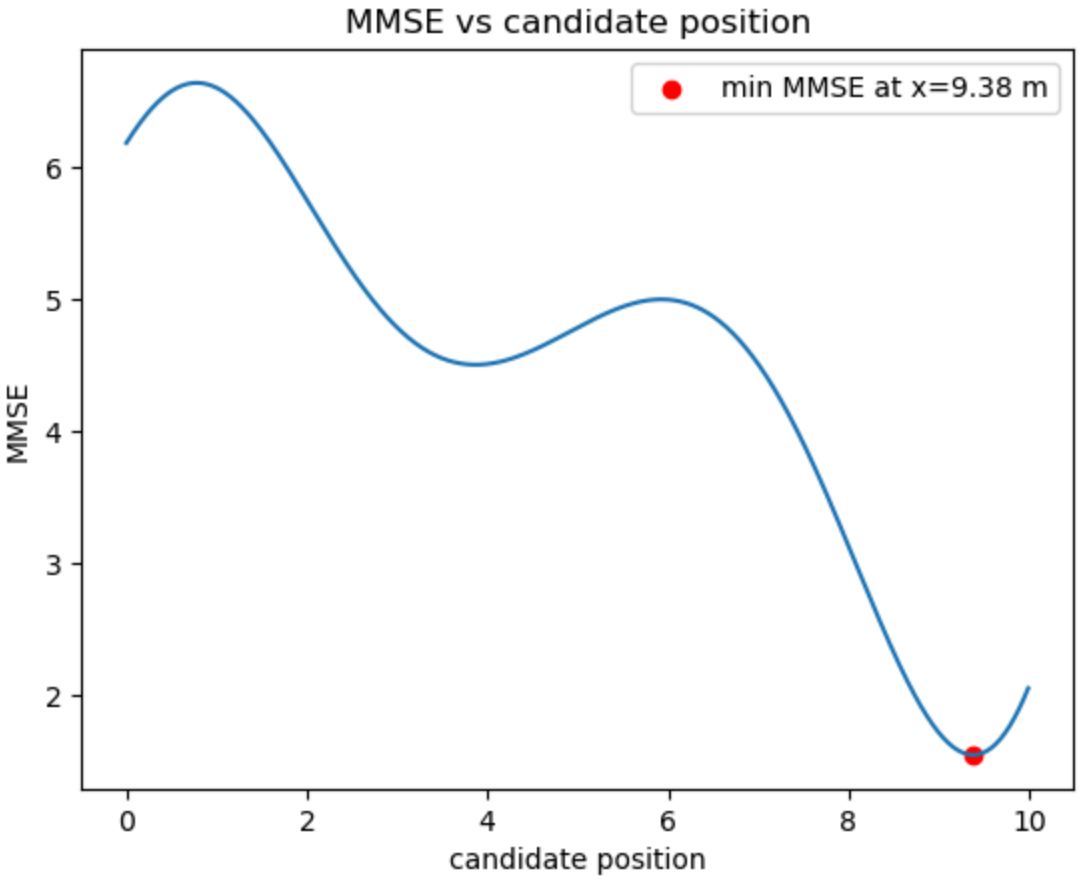
בשאלה 4 בדקתי את השפעת גודל חלון הזמן על השערוך.
בחרתי ב 0.2 , 0.5 , 2 שניות.



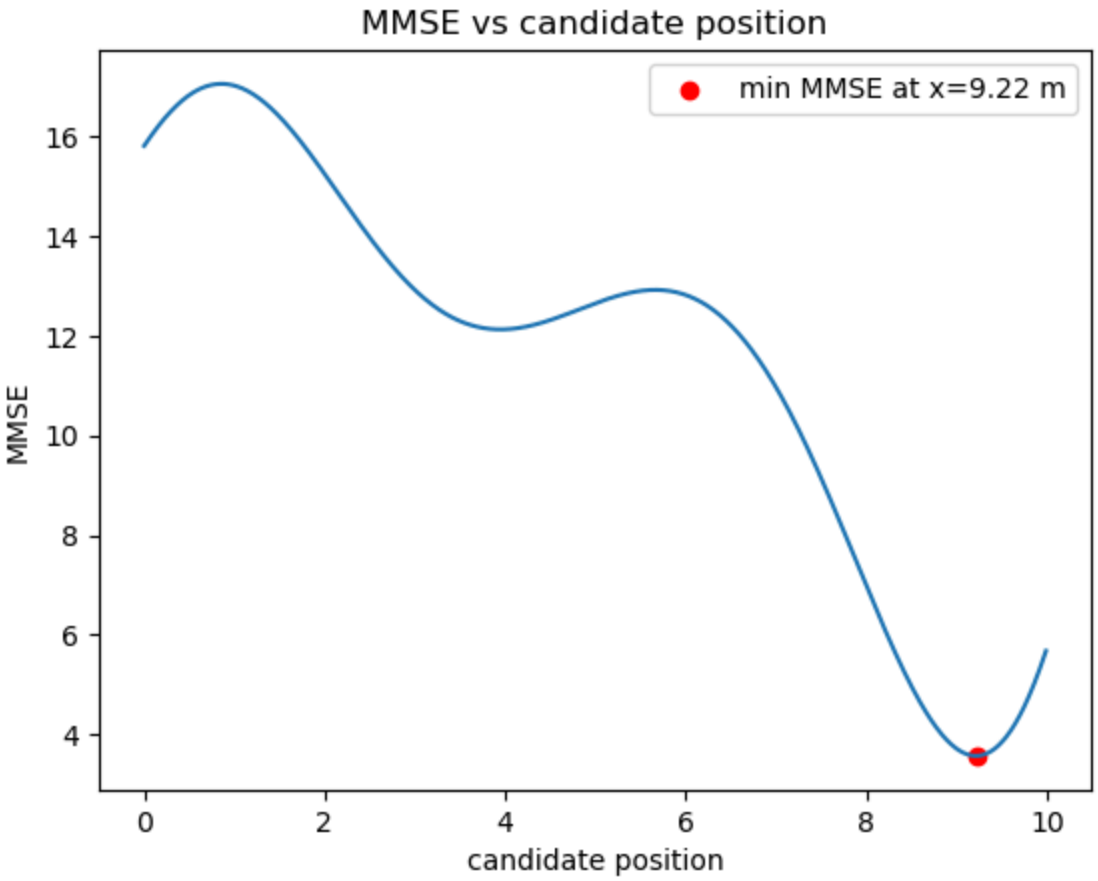
ניתן לראות בבירור שכמעט אין הבדל בין חלון זמן של חצי שניה לשניה הם כמעט חופפים.
לעומת זאת חלון זמן של 0.2 שנים מייצר לנו רעש וריצוד וחלון זמן של 2 שניות זה חלון זמן גדול מידי שמייצר כמות ערכים לא מספקת כדי לקבוע בדיוק את מיקום הדובר.

בנוסף בגרפי השגיאה ניתן לראות כי ככול שחלון הזמן קטן יותר כך סדר גדול השגיאה קטן יותר.

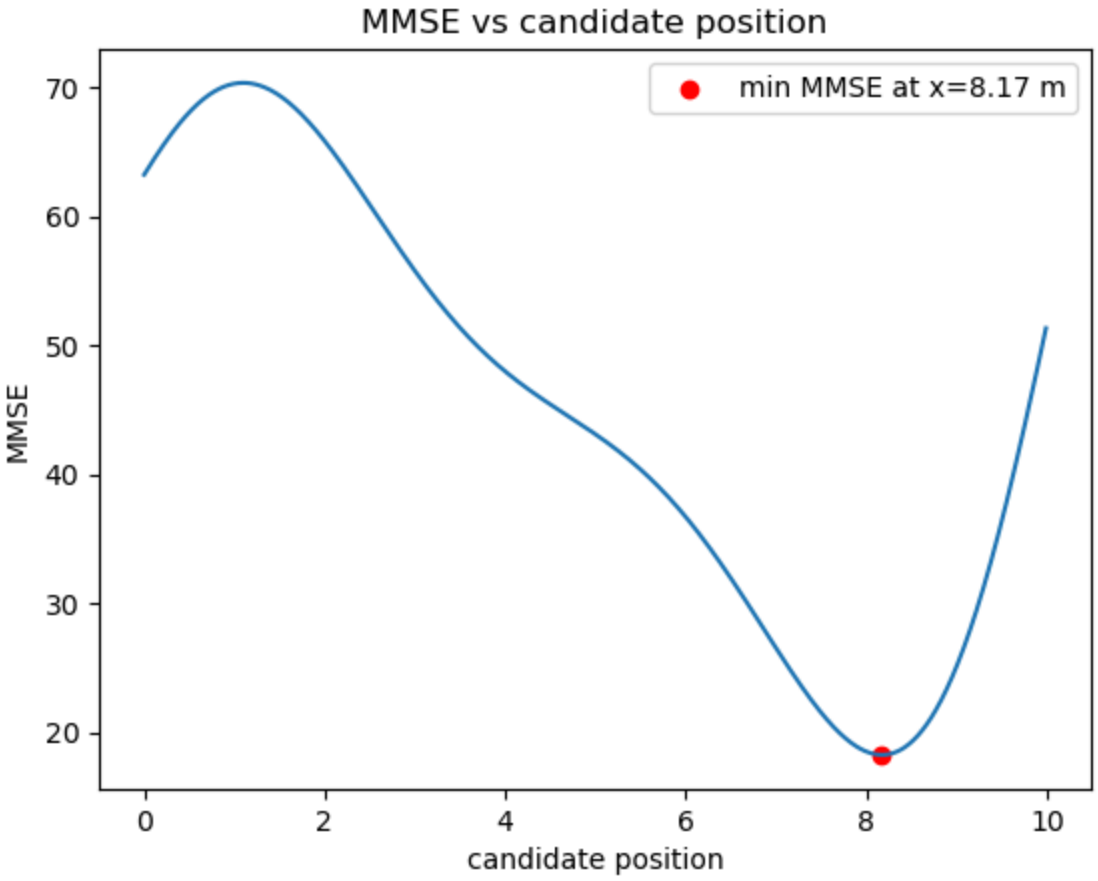
גרף השגיאה עבור חלון של 0.2:



גרף השגיאה עבור חלון של 0.5:



גרף השגיאה עבור חלון של 2:



שאלה 5:

דרך נוספת לפתרון הבעיה היא הוספת רכיב סינוס על כל פיצ'ר משמע:

$$F_{k,1} = cos(2\pi f_0 t \ - \phi_k(x_c))$$

$$F_{k,2} = sin(2\pi f_0 t \ - \phi_k(x_c))$$

ואז מטריצת הפיצ'רים תהיה :

$$\mathbf{X(x_c)} = \begin{bmatrix} F1,cos & F1,sin & 0 & 0 & 0 & 0 \\ 0 & 0 & F1,cos & F2,sin & 0 & 0 \\ 0 & 0 & F0 & 0 & F3,cos & F3,sin \end{bmatrix}$$

ווקטור המשקלים יהיה:

$$w = [w_1c,\ w_1s,\ w_2c,\ w_2s\ , w_3c,\ w_3s]$$

השילוב של קוסינוס וסינוס מייצר לנו ייצוג מדויק יותר של כל פאזה מכיוון שכל פונקציה מחזורית יכול להיות מיוצגת ע"י צירוף לינארי של סינוס וקוסינוס, ואצלנו האות הנקלט הוא אכן כזה.

$$s_k = a_k(x_s(t_n)) \bullet cos(2\pi f_0 t_n - \varphi_k) = w_1 cos(2\pi f_0 t_n) + w_2 sin(2\pi f_0 t_n)$$

$$w_1 = a_k(x_s(t_n))cos(\varphi_k)$$

$$w_2 = a_k(x_s(t_n))sin(\varphi_k)$$

לפי זהות : $cos(A-B) = cos(A)cos(B) + sin(A)sin(B)$

includes Linear RegressionLoss Function
using Engineering Study Copilot
main prompts and answers : Agent Conversation